Razorpay AI Buildathon 2026 | Track 03: AI Revenue Recovery

Senior Technical Pitch Strategy & 5-Minute Demonstration Master Plan

Project: Revenue Recovery Brain | **Author:** Himanshu | **Status:** 29/29 Tests Passing (100%)

Architecture: Multi-Modal Recovery Grid · Persisted SHA-256 Ledger · Bank Circuit Breakers · RBI FPC Gate

Contents

1 The 'Why': Real-World Problem & The Non-Technical Analogy

1.1 The Macro Financial Bleed in Indian Payments

Digital commerce in India runs at immense volume, yet revenue degrades systematically across four disconnected funnels:

- **1.2 Billion Monthly Payment Drops:** India processes over 15 billion monthly UPI transactions with an approximate 8% decline rate.
- **High-Friction Checkout Abandonment:** Mobile intent app mismatches and payment drop-offs account for 70%+ uncompleted shopping sessions.
- **Subscription Involuntary Churn:** RBI e-mandate regulations require explicit Additional Factor Authentication (AFA) for recurring charges exceeding ₹ 15,000, causing silent subscription churn.
- **73-Day B2B DSO & MSME Tax Clock:** Indian MSMEs endure an average 73-day Days Sales Outstanding (DSO). Under Section 43B(h) of the Income Tax Act, buyer invoices overdue past 45 days lose statutory income tax expense deductions.

The Non-Technical Analogy: The "Dumb Water Pump" vs. The "Smart Irrigation Grid"

Imagine a commercial building supplied by four distinct water pipelines:

1. **Pipe 1 (Retail Gateway):** The municipal water main connected to city reservoirs.
2. **Pipe 2 (Checkout Cart):** Bathroom faucets that tenants turn on and leave running.
3. **Pipe 3 (Subscription Mandates):** Automated monthly lawn sprinklers scheduled every 30 days.
4. **Pipe 4 (B2B Trade Invoices):** A large commercial water tanker delivering bulk water on credit.

The Traditional Dunning Trap: When water stops flowing in Pipe 1, conventional dunning tools act like a **dumb motorized pump**: they simply crank the pressure knob to 100% and pump repeatedly.

- If the municipal line is shut down for maintenance (an HDFC or SBI issuer switch outage), cranking the pump delivers zero water—it burns out the motor (wastes merchant API fees) and vibrates the building walls with loud alarms (spams the debtor).
- If the tenant's individual tank is empty (insufficient balance), blasting pressure into a closed valve accomplishes nothing.
- Worst of all, the landlord has no idea that the tenant whose kitchen tap failed also owes ₹ 85,000 for the commercial tanker downstairs!

Why Businesses Need Revenue Recovery Brain: Our engine replaces the dumb pump with an **intelligent hydraulic control grid**. In **<150ms**, it distinguishes between a **Technical Degradation (TD)** (switch outage → reroute or pause for NPCI off-peak) and a **Business Decline (BD)** (insufficient funds → respectful WhatsApp nudge timed to salary cycles). It unifies all four pipes under one customer risk profile and calculates Expected Net Recoverable Value (ENRV) before touching any banking rail.

2 The 'What': Trust Boundary Architecture & Production Engineering

In financial engineering, letting an unconstrained LLM directly invoke payment APIs is an unacceptable security risk. We establish an absolute separation between **probabilistic reasoning** (the investigator) and **deterministic policy authorization** (the police chief).

The Four Trust Boundaries (The Bank Branch Metaphor)

1. **1. The Bouncer (State Store & Write-Ahead Log Idempotency Guard):** Stands at the entrance. Ingests raw Razorpay webhooks, extracts the unique event key, and computes a SHA-256 fingerprint: SHA256(event_id ‖ payload_hash). It acquires an atomic lease lock in SQLite WAL mode. If 10 duplicate retries hit within 20 milliseconds, the Bouncer admits the first request and discards the other 9 with HTTP 409 Conflict.

2. **2. The Investigator (Diagnosis Engine & LLM Reasoning Core):** Operates with **read-only privileges**. It inspects error codes, bank responses, transaction amounts, and the unified cross-leak customer history in <150ms. It calculates counterfactual Expected Net Recoverable Value (ENRV) discounted by an 18% p.a. WACC benchmark. It proposes an intervention plan (e.g., "Hinglish Voice Call at 2:00 PM IST"), but possesses **zero execution authority**.

3. **3. The Police Chief (Policy Engine, Autonomy Envelope & Circuit Breaker):** Holds absolute veto power over the Investigator. It evaluates hardcoded regulatory and safety invariants that no LLM prompt can override:

   - **RBI Fair Practices Code:** Interventions between 7:00 PM and 8:00 AM IST are immediately vetoed and rescheduled to 9:00 AM.
   - **Dynamic Autonomy Envelope:** Capped at ₹ 25,000 in normal mode; automatically contracts to ₹ 5,000 if bank rails experience outages.
   - **Bank Circuit Breaker:** Monitors rolling Exponential Moving Average ($\alpha = 0.10$) across HDFC, SBI, ICICI, Axis, and NPCI. Trips if success rate drops below 30%.

4. **4. The Cashier (Executor & Persisted Cryptographic Audit Ledger):** Only acts when the Police Chief signs the execution permit. Dispatches official Razorpay Payment Links (`https://rzp.io/i/...`), triggers telephony workflows, and synchronously appends an immutable block to the SQLite SHA-256 hash sequence.

3 The Secret Weapon: Idempotency vs. LLM Wrapper Apps

Engineering Dimension	Basic LLM Wrapper Hackathon App	Razorpay Revenue Recovery Brain
Webhook Ingress	Assumes 1 webhook = 1 event.	Assumes at-least-once delivery; natively defends against concurrent replay spikes.
Concurrency Control	Zero mutexes. Concurrent requests spawn duplicate LLM agent instances.	Atomic millisecond lease locks in SQLite WAL mode with auto-expiring leases.
API Cost Exposure	Pays OpenAI $0.05 on every redundant network retry.	Intercepts duplicates at the edge (<1ms) with zero AI compute spend.
Debtor Experience	Spams customer with 3 duplicate SMS/WhatsApp links.	Strict exactly-once outbound customer communication guarantee.
Auditability	Mutable SQL rows (records can be edited or deleted).	Immutable SHA-256 hash chain (prev_hash → content_hash) verifiable offline.
Failure Filtering	Retries closed bank accounts and expired cards blindly.	Terminal failure filter halts futile retries on terminal codes (`ACCOUNT_CLOSED`, `CARD_STOLEN`).

Table 1: Production Infrastructure vs. Superficial AI Wrappers

4 The 'What's Next': High-Impact Polish Before the Deadline

To make the system visually undeniable on screen during the demonstration, we focus on three interactive elements:

1. **Interactive "10x Concurrency Sabotage Bomb" in Webhook Sandbox:** Add a dedicated button in the Webhook Playground: **"[SABOTAGE TEST] Blast 10x Concurrent Duplicate Webhooks"**. When clicked, it dispatches 10 parallel HTTP POST requests with identical event IDs. The dashboard immediately renders a race-condition breakdown:

   1 Winner (200 OK) | 9 Blocked (Idempotency Guard: 409 Conflict) | Latency: 14ms

2. **Live Bank Circuit Breaker ↔ Autonomy Envelope Contraction Toggle:** Add an interactive toggle card: **"Simulate HDFC Rail Outage (<30% SR)"**. Clicking it demonstrates the real-time wiring from `test_28`: the Autonomy Envelope badge dynamically contracts from `EXPANDED: ₹ 25,000 Cap` to a pulsing red `CONTRACTED: ₹ 5,000 Cap`.

3. **In-Browser One-Click Cryptographic Audit Inspector:** Add an "Audit Ledger Explorer" modal in the React dashboard that fetches `/api/audit-ledger/export`, computes the SHA-256 chain in client-side JavaScript, and displays a green shield: `100% Chain Integrity Verified (52 Blocks Checked)`.

5 The 5-Minute Video Demonstration Master Plan (Scene-by-Scene)

5.1 Pre-Recording Setup Checklist

- **Display Split-Screen:** Left 60% = React Operator Console (`http://localhost:5173`), Right 40% = Razorpay Merchant Test Portal (`dashboard.razorpay.com/app/payment-links`).
- **Terminal Tab:** Positioned in `backend/` with `python verify_ledger.py` pre-typed.
- **Telephony:** Test phone placed on desk on speakerphone.
- **Target Timing:** Exactly 4 minutes 50 seconds (10-second safety buffer).

5.2 Minute-by-Minute Script & Visual Execution

Scene 1: The Macro Problem & The Solution Thesis (0:00 – 0:40)

Screen View: React Command Center overview. Slowly scroll across live KPI cards: ₹ 8.7L at risk, ₹ 5.9L recovered, 68% recovery rate, 128ms average diagnostic latency.
Narration (Calm, Confident, Direct):

> "India processes 15 billion UPI transactions every month. About 8% fail—that is 1.2 billion declined transactions monthly. Today, almost every merchant responds to these failures the same way: blasting blind retries.
>
> That fails because a bank switch outage and a customer with insufficient balance require completely different handling. Meanwhile, cart drop-offs, subscription mandate failures, and 70-day overdue B2B invoices leak revenue through separate tools with zero shared intelligence.
>
> We built the Revenue Recovery Brain: a production-grade, regulatory-fluent recovery engine with sub-150ms diagnosis, hard RBI compliance gates, and cryptographic auditability."

Scene 2: The Concurrency Sabotage Test (0:40 – 1:25)

Screen View: Navigate to the Webhook Playground tab.
Action: Click **"Blast 10x Concurrent Duplicate Webhooks"** (or execute parallel curl test script).
Narration:

> "Before showing how we rescue revenue, let me show you why this is a production-level system and not an LLM wrapper. Payment gateways retry webhooks up to 3 times over 24 hours. Under network latency, duplicate deliveries happen constantly. If a system isn't idempotent, it calls an LLM 10 times, generates 10 duplicate payment links, and charges the customer twice.
>
> Watch: we just fired 10 simultaneous duplicate failure payloads across 10 threads. Notice the result: exactly 1 request was processed. The other 9 were rejected by our State Store Idempotency Guard with millisecond atomic leases. Zero duplicate links. Zero wasted compute."

Scene 3: Live Razorpay Split-Screen & Real Phone Rings (1:25 – 2:25)

Screen View: Split Screen: Left = Webhook Sandbox, Right = Real Razorpay Test Portal (pre-opened to Payment Links).
Action 1: Select "B2B Invoice Overdue · 38 Days" → Click **Dispatch Webhook**.
Action 2: Switch to Razorpay dashboard on right → Refresh. The live `plink_` ID appears immediately.
Action 3: Navigate to Hinglish Voice Agent tab → Enter test phone number → Click **Trigger Real Call**.
Action 4: Phone rings on camera → Answer on speakerphone: *"Namaskar! Kya main Rohit Mehta ji se baat kar sakta hoon? Main Recovery Brain se bol raha hoon..."*
Narration:

> "In 143 milliseconds, our engine diagnosed the root cause as cash flow strain and generated a recovery payment link. Look at the right screen: that `plink_` appeared in an authentic Razorpay test account. It is live, SMS-enabled, and settles to real banking rails.

For B2B invoices, emails get ignored. Our voice agent calls directly in Hinglish—the actual language of Indian commerce. Crucially, under RBI Master Directions, our agent is architecturally prohibited from asking for OTPs, CVVs, or UPI PINs. The customer pays exclusively through the authentic Razorpay link."

Scene 4: The Compliance Gate Visibly Refusing to Act (2:25 – 3:20)

Screen View: Navigate to the RBI Compliance tab.
Action 1: Set time to **9:30 PM IST** → Click **Trigger Intervention**. Card turns RED: `BLOCKED_TIME_WINDOW (RBI Fair Practices Code)`.
Action 2: Set time to **2:00 PM IST** → Click **Trigger Intervention**. Card turns GREEN: `ALLOWED`.
Narration:

"Most demo presentations only show the happy path. In fintech, an agent's ability to refuse to act is far more critical than its ability to act.

At 9:30 PM IST, our policy engine blocks the intervention immediately. Why? The RBI Fair Practices Code strictly bans recovery calls after 7:00 PM. The system automatically reschedules it for 9:00 AM tomorrow.

At 2:00 PM, the exact same intervention is approved. Our compliance engine enforces contact windows, 48-hour cool-offs after disputes, and an economic floor rule stopping any action where recovery cost exceeds value."

Scene 5: Bank Rail Outage & Cross-Leak Intelligence (3:20 – 4:10)

Screen View: Trigger "Simulate HDFC Outage" toggle, then open `/api/demo/unified-recovery-scenario`.
Narration:

"When an issuer fails, we don't retry blindly. Our Bank Circuit Breaker monitors rolling success rates across HDFC, SBI, ICICI, and NPCI. When HDFC falls below 30%, the breaker trips and automatically contracts our Autonomy Envelope from ₹ 25,000 down to ₹ 5,000.

And here is our biggest differentiator: Cross-Leak Unification. Customer Rohit Mehta has an overdue B2B invoice of ₹ 85,000 approaching the MSME Section 43B(h) tax deadline. When his personal retail checkout fails for ₹ 4,500, the system recognizes the correlated liquidity strain. It suppresses spamming him with duplicate WhatsApp messages and routes him to an executive restructuring workflow."

Scene 6: Standalone Cryptographic Audit Proof & Karpathy Close (4:10 – 5:00)

Screen View: Switch to terminal window.
Action: Press `Ctrl+C` to kill the FastAPI backend server. Run: `python verify_ledger.py`.
Result: All 49+ blocks verify to Genesis with `100% Chain Integrity | VERDICT: PASSED`.
Narration:

"Finally, financial recovery requires mathematical accountability. Every intent, compliance decision, and recovery event is chained into an immutable SHA-256 cryptographic ledger persisted in SQLite.

Notice that my server is completely shut down right now. I run our standalone, zero-dependency `verify_ledger.py` script directly against the database. It recalculates

the hash sequence from Genesis to Head: 100% integrity verified.

All 29 architectural verification tests pass. Built strictly under the Karpathy principle of simplicity, deterministic boundaries, and verifiable proof. Thank you."

6 Visual Demonstration Guide: The Concurrent Webhook Sabotage Test

6.1 The Scripted Python Sabotage Command

To execute the live 10-thread concurrency test from the terminal during your recording:

```
# One-liner to fire 10 simultaneous duplicate failure payloads across 10 threads
python -c "import threading, requests; [threading.Thread(target=lambda: print(requests.
    post('http://localhost:8000/api/webhooks/razorpay', json={'event': 'payment.failed',
    'payload': {'payment': {'entity': {'id': 'pay_race_101', 'amount': 150000, '
    error_code': 'BAD_REQUEST_ERROR'}}}}, headers={'X-Razorpay-Event-Id': '
    evt_race_unique_001'}).status_code)).start() for _ in range(10)]"
```

6.2 Terminal Output Displayed to Judges

```
200 OK          <-- Winner (First thread acquires atomic lease lock)
409 Conflict    <-- Duplicate Ignored (IdempotencyGuard blocked)
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
409 Conflict    <-- Duplicate Ignored
```

Key Takeaway for Razorpay Evaluators

This demonstrates that your platform utilizes atomic lease locking to defend against replay attacks and duplicate retries, eliminating duplicate customer charging and wasted LLM token expenses.

7 Visual Demonstration Guide: The React Operator Console & Audit Trail

When guiding judges through the dashboard, click on any recovered case row to open the **Case Detail Modal**, which displays three distinct verification views:

- **1. Telemetry & Diagnostic Chain:** Displays the detected root cause (`BD_INSUFFICIENT_FUNDS`), measured diagnostic latency (124ms), and error code classification.
- **2. Mathematical Decision Receipt:** Shows the counterfactual lift calculation:

  Gross: ₹ 5,000 | Natural: 8% | Intervention: 68% | ENRV Lift: +₹ 420.50

  Displays the cryptographic seal: `seal_7f8a12bc90de44...`
- **3. Cryptographic Block Inspector:** Displays the exact block sequence number, previous block hash, content payload, and SHA-256 block hash, proving permanent inclusion in the ledger.

8 Judges Defense & Technical Q&A Playbook

Q1: "Why use SQLite instead of PostgreSQL with distributed Redis locks?"

Your Answer: "For this hackathon reference implementation, our primary objective was 100% zero-dependency local reproducibility. Any evaluator can clone the repo and run all 29 tests immediately with zero Docker, PostgreSQL, or Redis configuration. SQLite in WAL mode with Python threading mutexes provides millisecond atomic lease guarantees.
In production, this architecture maps directly to PostgreSQL row-level locks (`SELECT ... FOR UPDATE NOWAIT`) and distributed Redis Redlock leases for multi-node FastAPI clusters, with Kafka handling incoming webhook ingress."

Q2: "Why didn't you use an LLM for the entire recovery workflow?"

Your Answer: "Using an LLM for policy enforcement or financial execution is an architectural anti-pattern. LLMs are probabilistic and hallucinate edge cases under pressure.
We use LLMs strictly where they excel: unstructured error text classification and colloquial Hinglish conversational dialogue. All policy boundaries—RBI calling hours, maximum attempt ceilings, and the Bank Circuit Breaker—are hard-coded, deterministic code invariants that no prompt can override."

Q3: "How does your ENRV model prevent negative-ROI interventions?"

Your Answer: "Traditional bots look only at recovery amount. We calculate counterfactual lift: we subtract the natural recovery probability (what would have recovered without any intervention) and apply continuous time-value discounting at an 18% p.a. WACC benchmark (the cost of capital for Indian SMEs).
If the projected ENRV is under ₹ 100, our economic floor rule aborts the action to prevent spending ₹ 15 on WhatsApp and telephony fees for an unviable return."

Document generated for Razorpay AI Buildathon 2026 · Track 03: AI Revenue Recovery
Built under the Karpathy Guidelines: Simplicity First, Surgical Execution, and Mathematical Truth.